

# Дилемма двойного агента

- **Ограничение по времени:** 12 minutes.
- **Хранилище:** 5 GB
- **Среда:** один GPU ( $\approx 16$  GB VRAM), без доступа к интернету
- **Размер решения:** `solution.ipynb`  $\leq 1$  MB
- **Бейзлайн:** 0

В национальном центре искусственного интеллекта в Астане анализируют фотографии две компьютерные модели — модель R (ResNet-18) и модель V (ViT-Tiny). В данный момент обе модели справляются идеально, демонстрируя ассигасу 100% и совпадая в предсказаниях для каждого изображения. Чтобы проверить, насколько в действительности различаются их умные «мозги», главный научный сотрудник ставит перед вами задачу: внести в каждую фотографию крошечные, почти незаметные изменения пикселей так, чтобы модель R и модель V полностью разошлись в своих предсказаниях.



## 1. Задача

Два предобученных классификатора изображений получают на вход одно и то же изображение. На наборе изображений, предоставленном в этой задаче, ассигасу обоих классификаторов 100%.

- **Модель R:** `torchvision.models.resnet18` (CNN, ResNet18).
- **Модель V:** `vit_tiny_patch16_224` от `timm` (Transformer, ViT-Tiny).

Ваша задача — создать небольшое изменение («возмущение») для каждого изображения так, чтобы две модели разошлись в своих предсказаниях. Для каждого изображения вы должны создать **два разных** возмущения:

- **Тип А:** после его добавления модель R по-прежнему классифицирует изображение правильно, но модель V классифицирует его неправильно.
- **Тип В:** после его добавления модель V по-прежнему классифицирует изображение правильно, но модель R классифицирует его неправильно.

Каждое возмущение должно быть достаточно *малым*, чтобы его было трудно заметить. За меньшие возмущения начисляется больше баллов (см. Section 5). Возмущение применяется непосредственно к исходному изображению в пиксельном пространстве.

## 2. Открытые данные

Вместе с задачей предоставляется набор изображений, разделённый на две выборки — `train` (100 изображений) и `test_public` (100 изображений), — каждая из которых содержит изображения различного разрешения. Все изображения относятся к 1000 классам ImageNet-1K, и обе модели, R и V, достигают точности 100% на обеих выборках.

Предоставляются следующие файлы:

```
train/images/*.png      # 100 картинок в формате PNG
train/labels.json       # корректный класс для каждой из train картинок
test_public/images/*.png # 100 картинок в формате PNG
test_public/labels.json  # корректный класс для каждой из test картинок
```

Во время тестирования ваша папка `dataset/test_public/` незаметно заменяется двумя скрытыми наборами изображений (`test_leaderboard_a` и `test_leaderboard_b`) для официального оценивания. Каждый из них содержит **100 изображений** в формате PNG и файл с метками.

**Примечание:** в этой задаче метки в тестовых датасетах доступны.

## 3. Формат выходных данных

Для каждого изображения вы должны создать два файла:

```
{index}_a.pt  # Возмущение (perturbation) типа А
{index}_b.pt  # Возмущение (perturbation) типа В
```

- $\{index\}$  (0, 1, 2, ...) соответствует имени изображения в датасетах.
- Каждый файл представляет собой один тензор, сохранённый с помощью `torch.save`. Его размер должен быть  $3 \times H \times W$ , где  $H$  и  $W$  соответствуют **исходному** разрешению этого изображения (не  $224 \times 224$ ).
- Код должен создавать только один ZIP-файл — `submission.zip`. Поместите все файлы `.pt` на верхний уровень ZIP-архива, без папок и без вложенных папок.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Ноутбук предупредит вас, если с форматом выходных данных возникнут какие-либо проблемы.

## 4. Ограничения

- **Модели:** вы должны использовать `torchvision.models.resnet18(pretrained=True)` и `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Использовать другие предобученные модели нельзя.
- **Предобработка входных изображений (принудительно применяется при оценивании):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. См. Section 3 в `baseline.ipynb` для подробностей.
- **Разрешение возмущения:** должно соответствовать **исходному** разрешению необработанного изображения (не  $224 \times 224$ ). Тензор добавляется к необработанному изображению до предобработки.
- **Формат output:** только файлы `.pt` — никаких PNG/JPG. Тензоры добавляются к необработанному изображению, а значения пикселей перед предобработкой обрезаются до диапазона  $[0, 1]$ .
- **Именованние файлов:** обычный список, строгий формат `{index}_a.pt` / `{index}_b.pt`. Внутри ZIP-архива не должно быть поддиректорий.
- **Библиотеки:** `torch`, `torchvision`, `timm`.

## 5. Оценивание

Итоговый результат вычисляется следующим образом. Пусть  $M$  — количество изображений в выборке,  $Score_A$  — количество успешных возмущений типа A, а  $Score_B$  — количество успешных возмущений типа B:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF — это функция, предназначенная для штрафования возмущений с высокой нормой и обладающая очень высокой чувствительностью вблизи верхней границы результата. Она ограничена диапазоном от 0.5 до 1. Полную реализацию можно увидеть в Section 8 файла `solution.ipynb`.

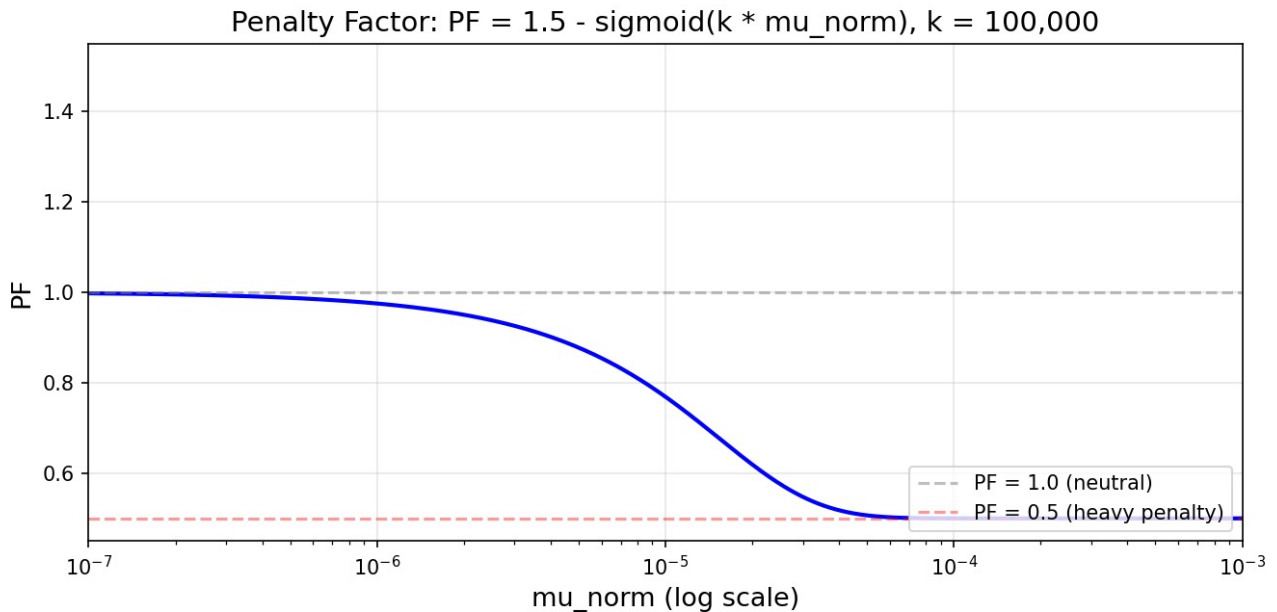


Рисунок: кривая штрафной функции.

## 6. Проверка посылки

В разделе 7 ноутбука `solution.ipynb` предусмотрены проверки, которые предупредят вас при наличии проблем с форматированием.

## 7. Локальное тестирование

`solution.ipynb` содержит полный работающий пример. Он загружает открытые данные, обе модели и официальный скрипт оценивания, а затем записывает ZIP-файл посылки. Прочитайте его перед началом работы.

## 8. Как отправить решение

- Сохраните свои изменения в `solution.ipynb`.
- Откройте вкладку Git на левой боковой панели JupyterLab.
- Выполните **Stage** для `solution.ipynb` (значок "+" рядом с ним).
- Введите сообщение коммита и нажмите **Commit**.
- Нажмите значок облака со стрелкой вверх, чтобы выполнить push.
- Вернитесь на эту страницу соревнования и нажмите **Submit**.

Отправьте ровно один файл с именем `solution.ipynb`.